# Příklad s prioritou procesu

## Zadání

> Vymyslete příklad systému hromadné obsluhy typu M/M/1, který bude demonstrovat použití priority procesu. Sestavte jeho model v Petriho síti a SIMLIBu.

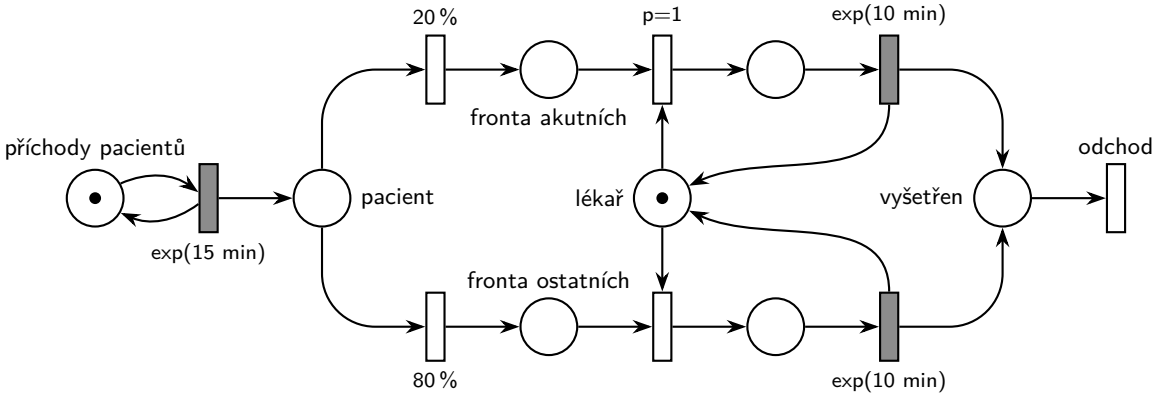
Podobné úlohy:

- Každá N-tá s prioritou procesu: prioritní je každá N-tá transakce
- Priorita procesu a obsluhy: rozdíl proti prioritě obsluhy
- Příklad s netrpělivou transakcí: stejný typ zadání „vymyslete příklad“

## Nápověda

- **Vymyšlený příklad:** ambulance s jedním lékařem. Pacienti přicházejí v intervalech `exp(15 min)`, 20 % z nich je akutních, vyšetření trvá `exp(10 min)`. Akutní pacienti mají vyšší prioritu procesu: když se lékař uvolní, jde na řadu akutní pacient před dříve příchozími ostatními. Probíhající vyšetření ale nepřeruší (to by byla priorita obsluhy, viz Priorita procesu a obsluhy).
- **Příchody** modeluje místo *příchody pacientů* ve smyčce s časovaným přechodem `exp(15 min)`.
- **Druh pacienta** určí pravděpodobnostní větvení hned po příchodu (20 % / 80 %). Větvicí přechody mají jen pravděpodobnost, priorita patří až na obsazení (jeden parametr na přechod, slajd 135).
- **Lékař** je zařízení s kapacitou 1: místo *lékař* s jednou značkou mezi oběma řadami. Obě obsazení si ji berou, oba časované přechody `exp(10 min)` ji po vyšetření vracejí.
- **Priorita procesu = pořadí ve frontě** (slajd 182, v SIMLIBu atribut `Priority`). Místo v Petriho síti neuchovává pořadí ani rozdíl mezi značkami. Proto má každý druh pacientů vlastní frontu a přednost se zapíše prioritou p=1 na obsazení lékaře z *fronty akutních*. Projeví se jen tehdy, když se lékař uvolní a čekají pacienti z obou front.
- **Vyšetření se nepřerušuje:** žádný přechod nebere značku z místa vyšetření, akutní pacient počká, až lékař dokončí rozpracované vyšetření.
- Obě obsazení by mohla vést do jednoho společného místa vyšetření, protože po začátku vyšetření už na druhu pacienta nezáleží. Dvě oddělená místa jsou tu kvůli přehlednosti a kvůli srovnání se sítí s prioritou obsluhy, kde nutná jsou.
- Pacient po vyšetření **opouští systém** přechodem *odchod*.

## Petriho síť




## Časté chyby

- Akutní pacient přeruší probíhající vyšetření. To je priorita obsluhy, ne priorita procesu.
- Jedna společná fronta a v ní „prioritní“ přechod. Místo nerozliší, která značka je akutní pacient.
- Pravděpodobnost i priorita na jednom přechodu (např. `20 %`, `p=1`). Přechod má jen jeden parametr.
- Priorita napsaná na hranu místo na přechod.
- V SIMLIBu `Seize(Lekar, 1)` místo `Priority = 1`. To je priorita obsluhy a akutní pacient by vyšetření přerušil.

## SIMLIB

Model je izomorfní k Petriho síti výše: každé místo, přechod a značka mají svůj protějšek v kódu. Obě fronty sítě jsou v SIMLIBu jedna vnitřní fronta zařízení, kterou atribut `Priority` řadí: akutní pacient se zařadí před všechny čekající ostatní.

```
// Ambulance s prioritou procesu – model v SIMLIB/C++
#include "simlib.h"

Facility Lekar("Lékař");                    // místo „lékař“ s jednou značkou
Histogram CekaniA("Čekání akutních pacientů [min]", 0, 5, 12);
Histogram CekaniO("Čekání ostatních pacientů [min]", 0, 10, 12);

class Pacient : public Process {            // transakce = značka, která prochází
 ↪ sítí
  void Behavior() {
    double prichod = Time;
    bool akutni = Random() < 0.2;           // okamžité přechody 20 % / 80 %
    if (akutni)
      Priority = 1;                        // priorita procesu = p=1 na obsazení z
 ↪ „fronty akutních“
    Seize(Lekar);                           // fronta + obsazení; akutní se řadí
 ↪ před ostatní
    (akutni ? CekaniA : CekaniO)(Time - prichod);
    Wait(Exponential(10));                  // časovaný přechod exp(10 min) ...
    Release(Lekar);                         // ... který vrací značku lékaři
  }                                         // konec Behavior = přechod „odchod“
};

class Prichody : public Event {             // místo „příchody pacientů“ ve smyčce
 ↪ s exp(15 min)
  void Behavior() {
    (new Pacient)->Activate();              // nová značka v místě „pacient“
```

```
    Activate(Time + Exponential(15));
  }
};

int main() {
  Init(0, 60 * 10000);                    // modelový čas v minutách: 10 000 hodin
  (new Prichody)->Activate();
  Run();
  Lekar.Output();
  CekaniA.Output();
  CekaniO.Output();
}
```

| Petriho síť | SIMLIB |
|---|---|
| *příchody pacientů* ⇄ `exp(15 min)` | událost `Prichody`, která se znovu plánuje za `Exponential(15)` |
| značka (pacient) | proces `Pacient` |
| `20 %` / `80 %` | `bool akutni = Random() < 0.2` |
| *fronta akutních* a *fronta ostatních*, obsazení s p=1 a p=0 | `Priority = 1` u akutních, pak `Seize(Lekar)`: vnitřní fronta zařízení je seřazená podle priority procesu |
| místo *lékař* (1 značka) | `Facility Lekar` |
| časovaný přechod `exp(10 min)` se sloučeným uvolněním | `Wait(Exponential(10)); Release(Lekar);` |
| žádný přechod nebere značku z vyšetření | `Seize` bez priority obsluhy: rozpracovanou obsluhu nikdo nepřeruší |
| přechod *odchod* | konec metody `Behavior()` |

**Kontrola simulací.** Petriho síť: *lékař* + vyšetřovaní pacienti = 1, všechny přechody se provádějí, fronty zůstávají omezené (`make check`). Akutních je asi 20 %, jejich fronta zůstává krátká, fronta ostatních je delší. SIMLIB model: v jednom běhu (10 000 h) byl lékař vytížen na 67 % (teoreticky $\rho = 10/15 = 2/3$). Akutní pacienti čekali v průměru 7,7 min a ostatní 22,4 min. Vzorce pro M/M/1 s prioritou bez přerušení dávají 7,7 min a 23,1 min, bez priorit by čekali všichni 20 min (`make simlib`).

## Původ a zdroje

**Původ:** zkouška 2019/20, 3. termín, skupina A (7. 2. 2020), otázka 2, 20 bodů.

**Zdroje:**

- `Fitcheats/Zkoušky z minulých let/2019-20 - 3. termín A.pdf`, str. 2 (fotka zadání, rámeček na odpověď je prázdný)

**Existující řešení:**

- K tomuto zadání žádné řešení není. Stejnou konstrukci (dvě fronty, obsazení s různou prioritou) kreslí **Studentská skripta 2015-16, str. 39** jako „slabou prioritu“.